

The Machine, Part 4: Data ani Aathvani

August 27, 2026

THE MACHINE

PART 4 OF 5: DATA आणि आठवणी

पाच PARTS चा नकाशा

PART 1	पैसा आणि MACHINE	झाला
PART 2	MACHINE शी बोलणं	झाला
PART 3	MACHINES चं जाळं	झाला
[तुम्ही इथे आहात]		
PART 4	DATA आणि आठवणी	storage, database, शोधणं
PART 5	जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

आधी Part 3 ची परीक्षा

पाच प्रश्न, Part 3 मधले. आधी स्वतः उत्तर द्या, मग खाली बघा. अडलात तर सोबतच्या chapter वर परत जा.

1. WhatsApp message थेट मित्राच्या phone ला का जात नाही? (5.1)
2. Video call मध्ये आवाज robotic का होतो, पण download झालेली file कधीच अर्धवट का नसते? (5.3)
3. कुलूप-icon (https) काय सांगतो आणि काय सांगत **नाही**? (5.6)
4. UPI ला "protocol" का म्हणावं, app का नाही? (5.4)
5. Internet चा मालक कोण आहे? (5.7)

उत्तर: 1. प्रत्येक message आधी company च्या server ला जातो, मग मित्राला; मध्ये तिसरी machine आहेच. 2. Call ला UDP-विचार: गेलेला क्षण परत मागवून उपयोग नाही, वेग महत्वाचा; file ला TCP: प्रत्येक तुकडा पोहोचलाच पाहिजे, किंमत म्हणून वेळ. 3. कुलूप सांगतं की **रस्ता** सुरक्षित आहे (मधला कोणी वाचू शकत नाही); पलीकडचा माणूस खरा आहे हे सांगत **नाही**. 4. कारण UPI म्हणजे खुले नियम आहेत, जे पाळणारं कुठलंही app/bank खेळात सामील होतं; NPCI ने n-squared ला protocol ने मारलं. 5. कोणीच नाही: हजारो मालकांची जाळी, समान नियमांनी जोडलेली; सेवांचे मालक आहेत, रस्त्याचा नाही.

या part मध्ये काय आहे

Part 3 ने message रस्त्यावर आणला. आता तो server वर पोहोचला. आणि तिथे जगातल्या प्रत्येक business चा सगळ्यात मोठा खजिना पडलेला आहे: **data**. Crores लोकांची खाती, orders, chats, आठवणी.

सात chapters: हा खजिना ठेवायचा कुठे (database), 10 million मधून एक नोंद डोळ्याच्या पापणीत शोधायची कशी (index), data चे आकार कुठले (shapes), दोघांनी एकाच क्षणी लिहिलं तर काय (transactions), कामाला रांगेत कसं लावायचं (queues), जे पुन्हा पुन्हा लागतं ते जवळ कसं ठेवायचं (cache), आणि सगळं हरवलं तर काय (backup).

हे तुमच्या business decision मध्ये कुठे येईल: तुमच्या business ची खरी मालमत्ता code नाही; **data** आहे (Part 3, Chapter 5.1: ज्याच्याकडे data, त्याच्याकडे ताकद). तो शोधता आला नाही तर धंदा हळू; तो चुकला तर ग्राहक गेला; तो हरवला तर धंदा **मेला**. हा part म्हणजे त्या मालमत्तेची राखण.

SECTION 6: लक्षात ठेवणं

सात chapters. Data ठेवण्यापासून तो वाचवण्यापर्यंत.

हे तुमच्या business decision मध्ये कुठे येईल: “App हळू का आहे” याचं उत्तर 10 पैकी 7 वेळा database मध्ये असतं. “Records जुळत नाहीत” ही तक्रार transactions च्या समजेशिवाय सुटत नाही. आणि “सगळा data गेला” ही बातमी ज्या company ची येते, ती बहुतेक पुन्हा उभी राहत नाही. या section नंतर तुम्ही engineer ला तीन नेमके प्रश्न विचारू शकाल: शोध कशावर चालतो, पैशाच्या नोंदी transaction मध्ये आहेत का, आणि backup **तपासला** आहे का.

Chapter 6.1 [SPINE]: विसरण्याची problem

Part 2 चं एक सत्य आठवा (Chapter 3.6): process मेली की तिची state जाते. जे वाचवायचं ते storage मध्ये लिहावं लागतं. मग सोपं वाटतं: file मध्ये लिहून टाका. संपलं?

दुकानाच्या वहीने सुरुवात करू, कारण file म्हणजे तेच: एक वही. छोट्या दुकानाला वही पुरते. आता दुकान वाढतंय. बघा वही कुठे कुठे तुटते:

नोंदी crores झाल्या. “पाटील यांची उधारी किती?” वही पान-पान चाळावी लागते. 10 million नोंदींची file म्हणजे तेच: उत्तरासाठी सगळं वाचणं.

लिहिणारे अनेक झाले. गल्ल्यावर तुम्ही, मागे भाऊ, online भाचा: तिघे **एकाच** वहीत एकाच वेळेस लिहू लागले तर? नोंदी एकमेकांवर चढतील, काही हरवतील. Files सोबत नेमकं हेच होतं: दोन programs नी एकाच वेळेस लिहिलं की भेळ.

प्रश्न अवघड झाले. “या महिन्यात 500 पेक्षा जास्त उधारी असलेले, गावाबाहेरचे ग्राहक कोण?” वहीत उत्तर **आहे**, पण विखुरलेलं. File पण असंच: data आहे, उत्तर काढायचं म्हणजे स्वतः recipe लिहा.

अर्धवट लिहिताना वीज गेली. नोंद अर्धी लिहिली गेली: “पाटील, 500 जमा...” आणि पुढचं गायब. आता वहीवर विश्वास कसा ठेवायचा?

चारही अडचणी एकत्र बघा, आणि design स्वतः सुचतं: वही **प्रत्येकाच्या हातात देणं** चूक आहे. एक वेगळा, कायम चालणारा program हवा, ज्याचं **एकच** काम आहे: data ठेवणं. सगळे programs त्याला **विनंती** करतील (“ही नोंद ठेव,” “हे शोधून दे”); वहीला हात फक्त तोच लावेल. तो रांग लावेल, अर्धवट नोंदी होऊच देणार नाही, शोध वेगवान ठेवेल.

या program चं नाव **database**. आणि लक्ष द्या: database म्हणजे file नाही; तो एक **server** आहे (Part 3 ची भाषा: देणारी machine), जो दिवसरात्र चालू असतो आणि data चा एकमेव दरबान असतो. सोसायटीचं office आठवा: कागदपत्रं सदस्यांच्या घरी नाही, office मध्ये असतात; पाहिजे ते secretary कडे **मागावं** लागतं, आणि नोंदी तोच करतो. नियम जाचक वाटतात, पण 500 सदस्यांची सोसायटी त्यांच्याशिवाय कोसळेल.

तुमच्या भोवतीचं जग आता या नजरेने बघा: bank चं प्रत्येक खातं एका database मध्ये आहे. Zomato ची प्रत्येक order, WhatsApp चा प्रत्येक group, IRCTC चं प्रत्येक ticket. जगातला जवळजवळ प्रत्येक business शेवटी एका database भोवती बांधलेला आहे, जसं जुनं गाव विहिरीभोवती.

इथे लोक काय चुकीचं समजतात

“Excel आहे ना, तोच आमचा database.” छोट्या कामाला खरंच पुरतो, आणि तिथे तो उत्तम आहे. पण वरच्या चार तुटी आठवा: Excel ची file एकाच वेळेस अनेकांनी लिहिली की भेळ होते, 1 million ओळींवर तो धापा टाकतो, आणि अर्धवट लिहिणं त्याला कळत नाही. हजारो businesses “Excel वर चाललय ना” म्हणत वाढले आणि एका दिवशी orders हरवायला लागल्या. Excel वही आहे; database दरबान आहे. दुकान केवढं आहे यावर निवड ठरते; पण फरक **माहीत** असणं founder चं काम आहे.

MAP वर

Database हा Level 3 च्या आतला एक अख्खा देश आहे: Oracle या एका company ने databases विकून decades राज्य केलं (आजही ~50 billion dollars ची company). मग open source आलं (Chapter 3.7): Postgres, MySQL फुकट झाले, आणि आता cloud companies तेच भाड्याने देतात (Part 5). आणि एक खोल गोष्ट: business विकला जातो तेव्हा खरेदीदार काय विकत घेतो? Building नाही, code ही नाही: **database** विकत घेतो: ग्राहक कोण, काय घेतात, किती देतात. Data हीच मालमत्ता, बाकी सगळं आवरण.

स्वतः बघा (5 मिनिटं)

तुमच्या bank च्या app मध्ये statement उघडा आणि filter लावा: फक्त “debit”, फक्त “या महिन्यातले”. अर्ध्या second मध्ये उत्तर आलं. आता कल्पना करा हेच काम एका कागदी वहीत करायचं: पान-पान, खूण करत. तुम्ही आता database ला काम करताना बघितलंत: crores नोंदींमधून, अटी लावून, क्षणात. पुढचा chapter सांगेल हे **कसं** शक्य आहे.

विचार करा

1. (derivation) भाजीवाल्याला वही पुरते; bank ला database लागतो. मधली रेषा नेमकी कुठे आहे? तीन प्रश्न बनवा, जे कुठल्याही business ला विचारून “वही की database” ठरवता येईल.

उत्तर: (1) **लिहिणारे किती?** एक माणूस: वही चालते. एकाच वेळेस अनेक (दुकानं, apps, branches): database. (2) **नोंदी किती?** हजारांत: काहीही चालेल. Millions मध्ये: शोधासाठी database. (3) **चुकीची किंमत काय?** भाजीची एक नोंद चुकली: 20 रुपये गेले. Bank ची एक नोंद चुकली: विश्वास गेला, license गेलं. जिथे चुकीची किंमत मोठी, तिथे दरबान लागतो. लक्षात घ्या: तिन्ही प्रश्न technology चे नाहीत, **business** चे आहेत. Tech ची निवड नेहमी business च्या प्रश्नांनी सुरू होते; हत्यारांनी नाही.

Chapter 6.2 [SPINE]: 10 million मधून एक शोधणं

मागच्या chapter चं वचन: database crores नोंदींमधून क्षणात शोधतो. आज ते वचन उघडून बघू, कारण आतली युक्ती technology तल्या सगळ्यात सुंदर युक्त्यांपैकी आहे, आणि ती तुम्हाला आधीच येते.

आधी problem नीट मोजा. 10 million (1 कोटी) नोंदी आहेत; “खाते क्रमांक 88231904 दाखवा.” सरळ रस्ता: पहिल्या नोंदीपासून बघत चला. सरासरी अर्ध्या नोंदी तपासाव्या लागतील: 5 million वाचनं, आणि disk वरून (Part 1: बाहेर जाणं महाग) म्हणजे कित्येक seconds, कधी minutes. प्रत्येक ग्राहकाला प्रत्येक प्रश्नाला minutes? धंदा मेला.

आता तुमच्या हातातली युक्ती: **शब्दकोश**. त्यात 1 lakh शब्द असतात. तुम्ही “पैसा” शोधताना पान-पान वाचता का? नाही. मधलं पान उघडता: “नमस्कार” दिसलं. “पैसा” त्याच्या **पुढे** येतो, म्हणजे मागचं अर्थ पुस्तक **फेकून दिलं**. उरलेल्याच्या मध्ये परत तेच. प्रत्येक पावलाला अर्थ शिल्लक.

हे किती जोरात आकसतं ते मोजा: 10 million -> 5 -> 2.5 -> ... फक्त **~23 पावलांत** एक नोंद उरते. 10 million चा शोध, 23 वाचनांत. Chapter 1.4 मध्ये doubling बघितलं होतं (प्रत्येक switch ने दुप्पट); हे त्याचंच उलटं: प्रत्येक पावलाला अर्थ. दुप्पट जितकं वेगाने वर जातं, अर्थ तितकंच वेगाने खाली येतं.

पण थांबा: ही युक्ती चालली कारण शब्दकोश **लावलेला** (sorted) आहे. Database च्या नोंदी येतात तशा पडलेल्या असतात. मग database काय करतो: मूळ नोंदी जिथे आहेत तिथे ठेवतो, आणि बाजूला एक **लावलेली यादी** बनवतो: फक्त खाते क्रमांक, क्रमाने, आणि प्रत्येकापुढे खूण “पूर्ण नोंद कुठे पडली आहे.” पुस्तकाच्या शेवटची सूची जशी: विषय अक्षर-क्रमाने, पुढे पान क्रमांक. या लावलेल्या यादीचं नाव **index**, आणि database ला विचारलेल्या प्रश्नाचं नाव **query**.

फुकट काहीच नसतं, मग index ची किंमत काय? दोन गोष्टी: **जागा** (यादी वेगळी ठेवावी लागते) आणि **लिहिण्याचा वेग** (नवी नोंद आली की मूळ वहीत पण लिहा आणि यादीतही योग्य जागी घुसवा; याद्या पाच असतील तर पाच ठिकाणी). म्हणून प्रत्येक column वर index लावत नाहीत; जिथे शोध वारंवार होतो तिथेच. दुकानदार उधारीची वेगळी यादी ठेवतो, पण “कोण कुठल्या रंगाचा shirt घेतो” ची यादी ठेवत नाही: यादीचं ओझं शोधाच्या फायद्यापेक्षा मोठं होईल.

आता जगाकडे बघा: Aadhaar system 1.4 billion लोकांमधून तुमची नोंद seconds मध्ये काढते: index. तुमच्या phone ची contact-यादी: नावं लावलेली आहेत, म्हणून शोध instant. आणि Google म्हणजे तर पूर्ण **web चा index**: त्यांचे programs जगाची पानं वाचून एक राक्षसी लावलेली यादी बनवतात (“हा शब्द या-या पानांवर”); तुमचा शोध त्या यादीत होतो, जगात नाही. एका index ने trillion-dollar company उभी आहे.

इथे लोक काय चुकीचं समजतात

”Computer वेगवान आहे, म्हणून शोध वेगवान.” नाही: **रचना** मुळे शोध वेगवान आहे. Index नसलेल्या column वर 10 million नोंदींची query द्या: तोच “वेगवान” computer minutes खाईल, app timeout होईल, ग्राहक जाईल. जगातल्या “app हळू आहे” तक्रारींपैकी मोठा हिस्सा म्हणजे कुठेतरी एक query विना-index चालते आहे. Machine ला दोष; गुन्हा रचनेचा.

MAP वर

रुपयाचा रस्ता: जो शोध वेगवान करतो तो कमावतो. प्रत्येक marketplace चं हृदय शोध आहे (Amazon वर वस्तू, Naukri वर उमेदवार); शोध 1 second हळू झाला की विक्री मोजून घटते (Chapter 2.3: Google/Amazon ची मोजमापं). म्हणून database tune करणारे consultants रोजंदारीवर लाखो घेतात, आणि “search जलद करा” एवढ्याच कामाच्या companies आहेत (Elasticsearch, Algolia). आणि आठवा: Google ने index नीट बनवला, एवढ्यावर जगातली सगळ्यात मोठी दुकानदारी उभी राहिली.

स्वतः बघा (5 मिनिटं)

Phone च्या contacts मध्ये कुठलंही नाव शोधा: instant, कारण नावांवर index आहे. आता gallery त जुना एखादा **विशिष्ट** photo शोधा (मागच्या दिवाळीचा तो एक): scroll, scroll, scroll... कारण “photo मधलं दृश्य” यावर index नाही; तुम्ही 5,000 नोंदी सरळ चाळताय. Index असणं आणि नसणं यातला फरक तुम्ही आता हाताने अनुभवलात. (आणि Google Photos “beach” शोधून देतं, कारण त्यांनी AI ने photos चे **वर्णन-index** बनवले आहेत: Book 2 ची झलक.)

विचार करा

1. (derivation) Index इतका जादुई आहे तर प्रत्येक column वर का लावू नये? आणि यातून काढा: कुठल्या business ला जास्त indexes परवडतात: जिथे **वाचणं** खूप आणि लिहिणं कमी (news site), की जिथे **लिहिणं** खूप (sensor चा data दर second)?

उत्तर: प्रत्येक index प्रत्येक **लिहिण्याला** महाग करतो: नवी नोंद आली की सगळ्या याद्या update. दहा indexes = एक insert दहा जागी. News site: दिवसातून शंभर लेख लिहिले, millions वेळा वाचले: indexes भरपूर लावा, वाचन राजा आहे. Sensor-data: दर second हजारो नोंदी लिहायच्या, वाचायच्या क्वचित: indexes कमीत कमी, नाहीतर लिहिणं बुडेल. धडा तोच जुना: प्रत्येक रचना एक **सौदा** आहे, आणि सौदा तुमच्या **कामाच्या आकारावर** ठरतो. “Best” रचना नसते; तुमच्या business ला जुळणारी रचना असते.

Chapter 6.3 [SPINE]: आठवणींचे वेगवेगळे आकार

घरातलं सामान तुम्ही कसं ठेवता, कधी लक्ष दिलंय? कपडे कपाटाच्या **खणांत**. किराणा **पिशव्यांत**, आणि मोठ्या पिशवीत छोट्या पिशव्या. किल्ल्या **खुंटीवर**, प्रत्येकीला नावाचा tag. तीन वेगळ्या पद्धती, आणि तुम्ही त्या विचार न करता निवडता, कारण **सामानाचा आकार पद्धत ठरवतो**.

Data चंही तसंच आहे. तीन मुख्य आकार:

खण (table). ओळी आणि रकाने. प्रत्येक ओळ = एक नोंद; प्रत्येक रकाना = एक गुण. Bank ची passbook हीच आहे: तारीख, तपशील, रक्कम, शिल्लक. जिथे नोंदी **एकसारख्या** आहेत आणि त्यांची **बेरीज-वजाबाकी, जोड-तुलना** करायची आहे, तिथे table राजा. Table शी बोलायची खास भाषा Part 2 मध्ये भेटली होती: **SQL** ("जिथे उधारी > 500, ती नावं दे"). Tables वाल्या database चं कुळ: Oracle, MySQL, **Postgres**.

पिशवी (document). काही नोंदी एकसारख्या नसतात; त्यांच्या आत पोट-नोंदी असतात. एक Instagram post बघा: मजकूर, photos ची यादी, आणि comments; प्रत्येक comment च्या आत replies, likes. हे खणांत बसवायचं म्हणजे ओढाताण; ही तर पिशवीत-पिशवी रचना आहे. अशा आकाराचं नाव **document**, आणि कुळ: MongoDB.

खुंटी (key-value). सगळ्यात सोपा आकार: एक नाव (key), त्यापुढे एक value. प्रश्न एकच विचारता येतो: "या नावाचं काय आहे?" पण म्हणूनच वीज-वेगवान. OTP ठेवायला (phone number -> "482913, 5 मिनिटांत संपेल"), login sessions ठेवायला. कुळ: Redis.

आता महत्त्वाचं वाक्य: **आकार कामावरून ठरतो, fashion वरून नाही**. एकाच Zomato च्या आत तिन्ही आहेत: पैशाचे व्यवहार tables मध्ये (हिशोबाला खण लागतात), restaurant चे menus documents मध्ये (प्रत्येकाची रचना वेगळी), आणि "आत्ता कोण login आहे" खुंटीवर (वेगासाठी). योग्य आकार निवडला की सगळं सोपं; चुकीचा निवडला की प्रत्येक दिवस ओढाताण, जशी किराणा कपाटाच्या खणांत ठेवण्याची.

(चौथा आकार जन्माला येतोय: AI साठी **vector**, जिथे "अर्थाने जवळचं" शोधता येतं. Book 2 मध्ये पूर्ण भेटेल.)

इथे लोक काय चुकीचं समजतात

दोन टोकांच्या चुका. पहिली: “SQL जुनं झालं, नवं Mongo-वगैरे वापरू.” हिशोब, पैसा, नोंदी-जुळणी: इथे tables चा 50 वर्षांचा राजेशाही अनुभव आहे; जुनं म्हणजे कालबाह्य नाही, जुनं म्हणजे **तावून-सुलाखलेलं** (Part 2 ची COBOL-गोष्ट आठवा). दुसरी: “एकच database सगळ्याला पुरेल.” लहान असताना खरंच पुरतो (आणि तेव्हा एकच ठेवावा!); वाढल्यावर आकार वेगळे होतात. चूक निवडीची नाही; निवड **का** केली हे माहीत नसण्याची असते.

MAP वर

आकारा-आकारावर companies उभ्या आहेत: tables (Oracle चं जुनं साम्राज्य, आता Postgres चं फुकट राज्य), documents (MongoDB, ~2 billion dollars वार्षिक धंदा), खुंटी (Redis). आणि आता AI च्या vector आकारासाठी नव्या companies चा सुळसुळाट: नवा आकार जन्मला की त्याच्याभोवती नवा बाजार जन्मतो, प्रत्येक वेळेस. तुमच्यासाठी खूण: जेव्हा data चा **नवा आकार** दिसायला लागतो, तेव्हा त्याच्या राखणदारीचा धंदा अजून रिकामा असतो.

स्वतः बघा (5 मिनिटं)

तुमच्या घरात एक फेरी मारा आणि तीन आकार शोधा: खण कुठे (कपाट, मसाल्याचा डबा), पिशवीत-पिशवी कुठे (किराणा, प्रवासाची bag), खुंटी-tag कुठे (किल्ल्या, चार्जर). प्रत्येक जागी विचारा: ही पद्धत **का**? खण = एकसारख्या गोष्टी, तुलना हवी. पिशवी = आत-आत रचना. खुंटी = एका नावावरून लगेच उचलायचं. Data चे आकार तुम्हाला आधीच येतात; फक्त नावं नवीन होती.

विचार करा

1. (derivation) एका bank ने खाती chat-सारख्या document आकारात ठेवली तर काय बिघडेल? (विचार करा: bank चे रोजचे प्रश्न कुठले असतात?)

उत्तर: Bank चे प्रश्न जोड-तुलनेचे असतात: “सगळ्या खात्यांची बेरीज जुळते का,” “या branch चे आजचे सगळे व्यवहार,” “500 पेक्षा जास्त शिल्लक असलेले.” हे प्रश्न खणांवर (tables) क्षणात चालतात, कारण प्रत्येक नोंदीचा ढाचा सारखा आहे आणि रकाने थेट जोडता येतात. Document आकारात प्रत्येक उत्तरासाठी पिशव्या उघडत बसावं लागेल, हळू आणि चुकांना जागा. आणि उलटंही तितकंच खरं: chat ला tables मध्ये कोंबणं म्हणजे प्रत्येक reply साठी नव्या खणांची ओढाताण. धडा: आधी **प्रश्न** लिहा, मग आकार निवडा. Data कसा दिसतो यापेक्षा, त्याला काय **विचारणार** हे design ठरवतं.

Chapter 6.4 [DEPTH]: दोघं एकाच क्षणी लिहितात तेव्हा

(DEPTH chapter. पण जर तुमच्या business मध्ये पैसा किंवा मोजकं सामान असेल: seats, stock, appointments, तर हा chapter तुमच्यासाठी SPINE च आहे.)

एक खरा प्रसंग: सिनेमाचं शेवटचं ticket शिल्लक आहे. तुम्ही आणि अजून कोणी, दोन वेगळ्या शहरांत, **एकाच क्षणी** “Book” दाबता. दोघांच्याही screens नी आत्ताच दाखवलं होतं: “1 seat शिल्लक.” काय व्हायला हवं? आणि काय होऊ **शकतं**?

आत काय घडतं ते हळू करून बघा. दोन्ही विनंत्या server वर पोहोचल्या. दोन्ही processes नी database ला विचारलं: “seat 14 रिकामी?” दोघांना उत्तर मिळालं: “हो” (कारण दोघांनी **एकाच वेळेस** विचारलं, कोणीच अजून लिहिलं नव्हतं). मग दोघांनी लिहिलं: “seat 14 माझी.” दोन तिकिटं, एक खुर्ची. सिनेमागृहात भांडण.

या घोट्याचं नाव **race** (शर्यत): दोन कामं एकाच data वर धावली, आणि निकाल कोण-आधी-पोहोचला या नशिबावर ठरला. Part 2 आठवा: OS पाळ्या इतक्या वेगाने फिरवतो की “एकाच वेळेस” रोजची गोष्ट आहे. मग इलाज?

पहिला इलाज तुम्ही रोज बघता: **रांग लावा**. लग्नात आहेर लिहिणारी वही एकच असते आणि धरणारा एकच; म्हणून नोंदी कधी चढत नाहीत. Database त याचं नाव **lock**: seat 14 ला हात लावण्याआधी कुंडी लावा; काम संपेपर्यंत दुसरा **थांबतो**. पहिल्याचं booking पूर्ण, कुंडी उघडली, दुसऱ्याला आता खरं उत्तर मिळतं: “seat गेली.” भांडण संपलं; किंमत: थोडी वाट.

पण अजून एक खोल problem उरली आहे. पैसे पाठवणं म्हणजे **दोन** नोंदी: तुमच्या खात्यातून 500 वजा, त्याच्या खात्यात 500 जमा. पहिली नोंद झाली आणि **त्याच क्षणी** वीज गेली तर? पैसे तुमच्याकडून गेले पण कोणाला मिळालेच नाहीत: system नेच चोरी केली. अर्धवट लिहिणं हे न-लिहिण्यापेक्षा वाईट आहे.

म्हणून database एक करार देतो, आणि हा करार या chapter चं हृदय आहे: **“ही कामं एक गट्टा आहेत: एकतर सगळी होतील, नाहीतर एकही नाही.”** या गट्ट्याचं नाव **transaction**. वजा+जमा एका transaction मध्ये बांधा; मध्ये वीज गेली तर database स्वतः अर्ध काम **उलटं फिरवतो** (rollback), जणू काही झालंच नव्हतं. रजिस्ट्रीची उपमा: घर विकणं म्हणजे पैसे देणं + नाव बदलणं; registrar office दोन्ही एकत्रच करतं. “पैसे दिले पण नाव जुनंच” ही अवस्था कायद्याने **अस्तित्वातच** येऊ दिली जात नाही.

आता तुमचा रोजचा अनुभव नव्याने वाचा: tatkal booking च्या वेळेस “payment झालं पण ticket नाही मिळालं; पैसे 3 दिवसांत परत” हे घडतं, तेव्हा तुम्ही transaction चा rollback अनुभवता: गट्टा पूर्ण होऊ शकला नाही, म्हणून system ने उलटं फिरवलं. चिडचिड होते, पण उलटी बाजू बघा: rollback नसता तर पैसे जाऊन ticket ही नाही, आणि कोणालाच पत्ता नाही.

इथे लोक काय चुकीचं समजतात

“Computer ला काय, एकाच वेळेस हजार कामं करतो.” करतो; पण **एकाच वहीवर** हजार हात एकाच वेळेस चालू शकत नाहीत; शेवटी नोंद एकच आहे. कुठेतरी रांग लागतेच: प्रश्न फक्त हा की रांग **design** केलेली आहे (lock, transaction) की नशिबावर सोडलेली (race). आणि दुसरी चूक: “असं क्वचितच होत असेल.” Flash sale, tatkal, exam form ची शेवटची तारीख: तुमच्या business चा सगळ्यात **महत्वाचा** क्षण हाच असतो जेव्हा सगळे एकदम येतात. Race नेमकी तेव्हाच धावते जेव्हा चूक सगळ्यात महाग असते.

MAP वर

हे जमवणारे लोक Level 3 मधले सगळ्यात महाग engineers आहेत: payment systems, stock exchanges, booking engines. कारण सूत्रात टाका: चुकीची किंमत प्रचंड (पैसा थेट), आणि जमणारे कमी (scarcity). NPCI च्या UPI ची रोजची **~600 million** transactions प्रत्येक वेळेस हा करार पाळतात: वजा-जमा नेहमी गळ्यात. एक race घोटाळा राष्ट्रीय बातमी होईल. जिथे “बरोबर असणं” हाच माल आहे, तिथे हा chapter म्हणजे पूर्ण business.

स्वतः बघा (5 मिनिटं)

पुढच्या वेळेस UPI payment करताना जाणीवपूर्वक बघा: “Processing...” चा क्षण. त्या क्षणात दोन banks च्या दोन databases मध्ये एक गट्टा बांधला जातोय: इथून वजा, तिथे जमा, दोन्हींची खात्री, मगच तुम्हाला हिरवी खूण. आणि कधी “pending” दिसलं तर आता घाबरू नका: गट्टा अर्धा अडकलाय आणि system तो पूर्ण करेल किंवा उलटा फिरवेल; तिसरी अवस्था करारानेच बंद आहे.

विचार करा

1. (derivation) दोन ATMs मधून, एकाच खात्यातून, एकाच क्षणी 800-800 काढण्याचा प्रयत्न झाला; खात्यात 1,000 आहेत. Race न होता हे कसं थांबवलं जातं, ते वरच्या दोन हत्यारांनी (lock, transaction) क्रमाने सांगा.

उत्तर: दोन्ही विनंत्या एकाच खात्याच्या नोंदीवर येतात; database त्या नोंदीला lock लावतो, म्हणजे एक विनंती **आधी** पूर्ण चालते: transaction सुरू -> शिल्लक तपासा (1,000) -> 800 वजा -> नोंद 200 -> गट्टा पूर्ण -> कुंडी उघडा. आता दुसरी विनंती आत येते, आणि तिला **ताजी** शिल्लक दिसते: 200. “पुरेसे पैसे नाहीत,” नकार. दोघांना 800 कधीच मिळत नाहीत, कारण “तपासणं आणि वजा करणं” हा गट्टा अर्ध्यात कोणी घुसू शकत नाही. हीच रचना seats, stock, appointments सगळीकडे: **मोजकी वस्तू + अनेक हात = lock + transaction**. तुमच्या business मध्ये मोजकं काय आहे, हे आत्ताच लिहून ठेवा; तिथेच ही रचना लागेल.

Chapter 6.5 [DEPTH]: कामाला रांगेत लावणं

(DEPTH chapter. लहान आहे, आणि यानंतर “हे काम आत्ता, ते नंतर” ही design ची नजर मिळते: ती founder ची नजर आहे.)

Zomato वर order पडली. आता किती कामं करायची आहेत मोजा: पैसे घ्या, order नोंदवा, restaurant ला कळवा, delivery वाल्याला शोधा, तुम्हाला SMS पाठवा, email पाठवा, loyalty points द्या, हिशोबाच्या वहीत नोंद करा. आठ कामं.

सोपं design: सगळी आठ कामं **आत्ता** करा, मग user ला “order झाली” दाखवा. काय बिघडेल? User थांबलेला आहे: आठ कामं x प्रत्येकाचा वेळ = फिरणारं चक्र, आणि Chapter 2.3 ची आठवण: उशीर = गेलेला ग्राहक. आणि वाईट: SMS पाठवणारी system **पडली** असेल तर? आठपैकी एक काम अडलं म्हणून पूर्ण order अडली? SMS साठी धंदा थांबवायचा?

आता हॉटेलचा waiter बघा; त्याला हे design आधीच येतं. तो तुमची order घेतो, kitchen च्या खिडकीत **चिठ्ठी लावतो**, आणि लगेच पुढच्या table कडे जातो. तो स्वयंपाक होईपर्यंत उभा राहत नाही. चिठ्ठ्या रांगेत लागतात; cooks आपल्या वेगाने त्या उचलतात. Waiter चं काम: मागणी **नोंदवणं**. Kitchen चं काम: ती **पूर्ण करणं**. दोघांचे वेग वेगळे, आणि म्हणूनच दोघांना जोडणारी **रांग** मध्ये आहे.

Software मध्ये या रांगेचं नाव **queue**, आणि या पद्धतीचं नाव **async** (आत्ता नोंदवा, नंतर करा). Order च्या आठ कामांना आता दोन रंगांत वाटा:

आत्ताच झालीच पाहिजेत: पैसे घेणं आणि order नोंदवणं (हे मागच्या chapter चा transaction-गट्टा). हे झालं की user ला हिरवी खूण: “Order झाली!” वेळ: 1-2 seconds.

नंतर चालतील: SMS, email, points, हिशोब. यांच्या चिठ्ठ्या रांगेत टाकल्या. वेगळे कामगार-programs (workers) त्या सावकाश उचलतात. SMS system पडली? चिठ्ठ्या रांगेत **साठून राहतात**; system उठली की निघतात. User ला पत्ताही नाही; त्याची order कधीच अडली नाही.

ही रचना तुमच्या रोजच्या अनुभवात उघड दिसते: WhatsApp वर message पाठवताना **घड्याळ-icon** दिसतं (तुमच्या phone च्या रांगेत आहे), मग एक tick (server च्या रांगेत पोहोचला), मग दोन (मित्राला मिळाला). “पाठवलं” म्हणजे “पोहोचलं” नाही; “रांगेत नोंदवलं.” आणि म्हणूनच network नसतानाही तुम्ही message “पाठवू” शकता: रांग वाट बघते. Diwali च्या मध्यरात्री सगळ्यांचे greetings उशिरा पोहोचतात ते म्हणूनच: रांगा भरलेल्या; पण **हरवत** कोणीच नाही.

इथे लोक काय चुकीचं समजतात

“चांगलं system म्हणजे सगळं realtime.” उलटं आहे: चांगलं system म्हणजे **नेमकं** काय realtime हवं हे माहीत असलेलं. सगळंच आत्ता करायचं तर सगळ्यांची ताकद सगळ्या क्षणी लागते: खर्च टोकाचा, आणि एक भाग पडला की सगळं पडतं. सरकारी office जेव्हा “अर्ज घेतला, पावती घ्या, निकाल टपालाने येईल” म्हणतं, तेव्हा ते (नकळत) उत्तम queue design आहे: घेणं वेगवान, करणं सावकाश. चीड येते ती रांग **किती लांब** आहे याची; रांग **असण्याची** नाही.

MAP वर

रुपयाचा रस्ता: रांगा चालवणं हा स्वतः infra-धंदा आहे (Kafka, RabbitMQ, cloud च्या queue-सेवा). पण तुमच्यासाठी मोठी गोष्ट खर्चाची आहे: “आत्ताच” ची ताकद **सगळ्यात महाग** असते; “नंतर चालेल” ची स्वस्त, कारण ती सावकाश, रात्री, स्वस्त machines वर चालवता येते. म्हणून product design करताना प्रत्येक feature ला विचारा: हे user समोर उभं राहून व्हायला हवं, की चिठ्ठी लावून? या एका प्रश्नाने server चं bill दुप्पट-अर्धं होतं. (Part 5 मध्ये हा हिशोब पूर्ण भेटेल.)

स्वतः बघा (5 मिनिटं)

Phone airplane mode वर टाका आणि कोणाला तरी WhatsApp message लिहा. घड्याळ-icon दिसेल: तुमची चिठ्ठी रांगेत आहे, वाट बघते आहे. आता airplane mode काढा: tick, मग दोन ticks, आपोआप. तुम्ही आत्ता एक queue पूर्ण चक्र चालताना बघितली: नोंदवलं, थांबलं, जमलं तेव्हा झालं, हरवलं काहीच नाही.

विचार करा

1. (derivation) Cinema ticket booking system design करताय. ही कामं दोन रंगांत वाटा: seat अडवणं, पैसे घेणं, ticket चा QR बनवणं, SMS पाठवणं, “आवडेल असे shows” सुचवणं. कुठलं आत्ता, कुठलं रांगेत? आणि एक काम **मुद्दाम** अवघड आहे: QR कुठे टाकाल?

उत्तर: Seat अडवणं + पैसे: आत्ताच, एका transaction-गठ्यात (Chapter 6.4: मोजकी वस्तू + अनेक हात). SMS आणि सुचवणं: रांगेत, उशीर चालतो. QR मधली गोम: user ला ticket **लगेच** दिसायला हवं (theatre ला निघालाय!), म्हणजे “नंतर चालेल” म्हणता येत नाही; पण QR बनवणं म्हणजे फक्त एक चित्र काढणं, त्यात मोजकं काही अडवायचं नाही, म्हणून transaction मध्ये कोंबूही नये. उत्तर: transaction पूर्ण होताच, रांगेच्या **सगळ्यात पुढच्या** टोकाला, आणि screen वर “ticket तयार होतंय...” ची दोन seconds. धडा: “आत्ता vs नंतर” हे दोनच रंग नाहीत; “किती नंतर चालेल” ही एक **मोजपट्टी** आहे, आणि design म्हणजे प्रत्येक कामाला तिच्यावर योग्य जागी ठेवणं.

Chapter 6.6 [DEPTH]: जे पुन्हा पुन्हा लागतं ते जवळ ठेवा

(DEPTH chapter. “Cache” हा शब्द Part 3 मध्ये (DNS) आणि Part 1 मध्ये (मंडईची फेरी) डोकावून गेला होता. आज त्याचा पूर्ण chapter, कारण Part 5 च्या खर्चाच्या हिशोबाचा हा पाया आहे.)

एक प्रसंग: India-Pakistan match चालू आहे. 100 million लोक दर second ला score विचारतायत. आणि score आहे database मधल्या **एका** नोंदीत. 100 million queries, एकाच नोंदीवर, दर second? कुठलाही database जळून जाईल.

पण थांबा: 100 million लोकांना उत्तर **एकच** द्यायचंय: “IND 287/4.” प्रश्न एक, उत्तर एक, विचारणारे 100 million. मग database पर्यंत 100 million वेळा जाण्यात अर्थ काय? **एकदा** जा, उत्तर लिहून घ्या, आणि पुढच्या सगळ्यांना **लिहिलेलं** दाखवा. Score बदलला की नवं आणा.

हीच ती युक्ती जिचं नाव **cache**: महाग रस्त्याने मिळवलेलं उत्तर जवळ ठेवणं, पुन्हा-पुन्हा लागणार म्हणून. तुमच्या घरात ती आधीच आहे: विहीर लांब आहे म्हणून घरात **माठ** भरून ठेवता; प्रत्येक घोटाला विहिरीवर जात नाही. Cook सकाळी **एकदाच** मंडई करतो (Chapter 2.3); दुकानदार गल्ल्याजवळ **चिल्लर** ठेवतो, तिजोरी मागच्या खोलीत. प्रत्येक वेळेस रचना तीच: लांबचं महाग, जवळचं स्वस्त, म्हणून वापरातलं जवळ आणा.

Machine च्या जगात ही रचना **प्रत्येक थरावर** आहे, एकात एक: तुमच्या phone मध्ये (app च्या images साठवलेल्या: airplane mode t chats दिसतात त्या इथूनच), server च्या समोर (score सारखी तयार उत्तरं, Redis सारख्या खुंटी-store मध्ये: Chapter 6.3), database च्या आत (गरम नोंदी RAM च्या मेजावर: Chapter 2.5), आणि CPU च्या आतही. Part 1 चं सूत्र आठवा: आतलं nanosecond, बाहेरचं millisecond, फरक 1 million पट. Cache म्हणजे तो फरक **खिशात घालणं**.

पण फुकट काहीच नाही. Cache ची किंमत एका शब्दात: **शिळेपणा**. माठातलं पाणी कालचं असू शकतं. Score cache केला आणि wicket पडली: काही seconds लोकांना जुना score दिसतो. आता design चा खरा प्रश्न आला: **किती जुनं चालेल?** Cricket score: 2-3 seconds चालतं, कोणाचं काही बिघडत नाही, आणि खर्च 100 million पर्तींनी वाचतो. News चा मथळा: minute चालेल. Bank balance दाखवणं: काही seconds कदाचित; पण **व्यवहार करताना** शिल्लक तपासणं: शिळं **कधीच** चालणार नाही; तिथे थेट database, transaction च्या कुंडीसह (Chapter 6.4). एकाच app मध्ये दोन्ही असतात, आणि कुठे कुठलं हे **ठरवणं** म्हणजेच design.

इथे लोक काय चुकीचं समजतात

”Data नेहमी ताजाच हवा.” ऐकायला छान; प्रत्यक्षात 90% कामांत काही seconds चा शिळेपणा कोणाच्या लक्षातही येत नाही, आणि तिथेच खर्च 10-100 पट घटतो. उलटी चूकही आहे: cache केलं आणि **विसरले**: जुनी किंमत दाखवली गेली, ग्राहकाने जुन्या भावात order केली, भांडण. Cache “कधी रिकामा करायचा” (invalidation) हा software मधल्या सगळ्यात कुप्रसिद्ध अवघड प्रश्नांपैकी आहे. म्हणून नियम: cache लावताना **त्याच क्षणी** लिहा: हे उत्तर किती जुनं चालेल, आणि शिळं दिसलं तर काय बिघडेल. उत्तर “काहीच नाही” असेल तरच लावा.

MAP वर

रुपयाचा रस्ता: Hotstar cricket च्या दिवशी crores एकाच वेळेस बघतात; ते cache शिवाय अशक्य आणि परवडणारंही नाही: एक उत्तर बनतं, crores ना वाटलं जातं; प्रति-प्रेक्षक खर्च जवळजवळ शून्य (leverage चा निखळ अवतार). आणि हीच कल्पना जगभर पसरवलेली एक अख्खी industry आहे: **CDN** (Cloudflare, Akamai): तुमच्या site च्या प्रती जगभरच्या शहरांमध्ये cache करून ठेवणारी. Part 5 मध्ये ती पूर्ण भेटेल. खूण लक्षात ठेवा: **जो एकच उत्तर लाखोंना विकतो, तो cache वर उभा आहे.**

स्वतः बघा (5 मिनिटं)

कुठलंही जड app (Instagram/YouTube) उघडा, एक post/video बघा, बंद करा. आता airplane mode लावून पुन्हा उघडा: आता बघितलेलं बरंचसं **दिसतं**, नवं काही येत नाही. जे दिसलं तो तुमच्या phone मधला cache आहे: लांबून आणलेलं, जवळ ठेवलेलं. आणि आता तुम्हाला तुमच्या phone चं “storage भरलं: cache साफ करा” हे बटणही कळलं: ती जवळ ठेवलेली उत्तरं फेकणं, जागेसाठी.

विचार करा

1. (derivation) तुमच्या दुकानाच्या app मध्ये दोन आकडे दिसतात: वस्तूची **किंमत** आणि “फक्त 2 शिल्लक!” हा **stock**. कुठला cache कराल, कुठला नाही, आणि का? (Chapter 6.4 ची आठवण ठेवा.)

उत्तर: किंमत: cache करा (minute-दोन minutes); ती क्वचित बदलते, आणि बदलल्यावर cache रिकामा करायचा नियम ठेवा. Stock **दाखवणं:** थोडं शिळं चाललं (“2 शिल्लक” प्रत्यक्षात 1 झालं असेल: फार तर उत्सुकता वाढेल). पण stock **वजा करणं**, म्हणजे order घेताना: कधीच cache नाही; ते Chapter 6.4 चं मोजक्या वस्तूचं युद्ध आहे: थेट database, lock, transaction. एकाच आकड्याचे दोन वापर, दोन नियम. हीच या part ची एकत्रित शिकवण: **दाखवणं स्वस्त आणि शिळं चालतं; बदलणं महाग आणि ताजंच लागतं.** हे वाक्य design च्या अर्ध्या चुका थांबवतं.

Chapter 6.7 [SPINE]: Data हरवला तर?

आधी एक सत्य जे कोणी सांगत नाही: **disks मरतात**. नक्की मरतात. प्रत्येक hard disk/SSD चं एक आयुष्य आहे; datacenter मध्ये रोज कुठली ना कुठली मरते. आणि disk एकटी मारेकरी नाही: building ला आग, पूर, चुकून `delete` दाबणारा माणूस (सगळ्यात मोठा मारेकरी हाच), आणि ransomware: तुमचाच data कुलूपबंद करून खंडणी मागणारे चोर.

आता तुमच्या business चा विचार करा: Part 4 ने आत्तापर्यंत सांगितलं की data हीच मालमत्ता. मग प्रश्न “disk मेली तर” नाही; “disk मेली तेव्हा” आहे. उत्तर माणसाला हजारो वर्ष माहीत आहे: सोनं एका जागी ठेवत नाहीत. थोडं घरात, बाकी locker मध्ये; महत्त्वाच्या कागदपत्रांच्या प्रती माहेरी. **प्रती (copies)**. पण प्रतींमध्ये दोन वेगळ्या जाती आहेत, आणि त्या मिसळणं ही जीवघेणी चूक आहे:

जिवंत प्रत (replication). दुसरी machine, जिच्यावर प्रत्येक नोंद त्याच क्षणी पुन्हा लिहिली जाते. मुख्य मेली, ही लगेच पुढे येते: धंदा थांबत नाही. दुकानाचा भाऊ जसा: गळ्यावरचा आजारी पडला तरी दुकान उघडं. **पण**: चुकून delete केलंत, तर भाऊ **तेही** इमाने इतबारे पुन्हा करतो. जिवंत प्रत चुकीपासून वाचवत **नाही**; ती चूकही copy करते.

काळ-प्रत (backup). काल रात्रीचा (किंवा दर तासाचा) पूर्ण photo, वेगळ्या जागी ठेवलेला, आणि मुख्य म्हणजे **मागे वळता येणारं**: “शुक्रवार रात्रीसारखं करून दे.” चुकून delete, ransomware, software च्या bug ने बिघडवलेला data: या सगळ्यांवर उत्तर फक्त हे. जुनी रोजकीर्द जशी: आजची वही जळाली तरी कालची शिल्लक नोंदलेली आहे.

दोन्ही लागतात, कारण शत्रू वेगळे आहेत: machine मरण्यावर replication; माणूस/चोर चुकण्यावर backup. आणि ठेवायचं कसं, याचा जगमान्य नियम लक्षात ठेवण्याइतका सोपा आहे: **3-2-1**. एकूण **3** प्रती; **2** वेगळ्या प्रकारच्या जागांवर; **1** दुसऱ्या **गावात** (building जळाली तरी गाव जळत नाही).

आणि आता या chapter ची सगळ्यात महत्त्वाची ओळ, अधोरेखित करा: **जो backup परत आणून बघितला नाही, तो backup नाहीच**. जगात रोज कुठेतरी हे नाटक घडतं: “backup आहे ना” म्हणत सगळे निश्चित; संकट आलं; backup उघडला; तो रिकामा/अर्धा/जुनाट निघाला: घेणारी recipe महिन्यांपूर्वी चुपचाप बिघडली होती, आणि कोणी **तपासलंच** नव्हतं. Fire drill जसं: नळी आहे म्हणून खात्री नाही; पाणी **येतं** का हे दर महिन्याला बघावं लागतं. Restore ची rehearsal हा backup चा भाग आहे, extra नाही.

तुमच्या स्वतःच्या आयुष्यात हे सगळं आधीच आहे: phone हरवला, नवा घेतला, Google Photos मधून सगळे photos परत आले. ते replication + backup तुमच्यासाठी कोणीतरी चालवत होतं, महिन्याच्या 130 रुपयांत (Chapter 2.1 चं भाडं). आता तेच तुमच्या business साठी जाणीवपूर्वक उभं करायचं आहे.

इथे लोक काय चुकीचं समजतात

”Cloud मध्ये आहे, म्हणजे कायमचं सुरक्षित.” Cloud म्हणजेही कोणाच्यातरी building मधल्या disks च (Part 5); त्या मरतात, accounts बंद होतात, आणि ransomware तुमच्या laptop वरून cloud च्या sync मध्येही चढतो (चूक copy होते: replication ची जात!). आणि दुसरी, engineers चीही चूक: “replication आहे ना, मग backup कशाला”: आता तुम्हाला माहीत आहे, त्या दोन वेगळ्या जाती आहेत, वेगळ्या शत्रूंसाठी. दोन्ही प्रश्न विचारायला शिका: “मुख्य मेली तर किती वेळात उभे?” आणि “चुकून पुसलं तर किती मागे जाता येतं?”

MAP वर

रुपयाचा रस्ता: backup/recovery ही स्वतः एक industry आहे (Veeam, Druva: ~billions revenue), आणि विम्यासारखी विकली जाते: रोज खर्च वाटतो, एका दिवशी company **वाचवते**. आकडे बोलतात: ज्या छोट्या companies चा data मोठ्या प्रमाणात जातो, त्यांतल्या बहुतेक वर्षभरात बंद पडतात; data गेला की Level 2 च मरतो, Level 3 ची चूक असूनही. आणि उलट बाजू: ransomware टोळ्या खंडणी **धंदा** म्हणून चालवतात; त्यांचा पूर्ण खेळच “यांच्याकडे तपासलेला backup नसेल” या bet वर आहे. तुमचा backup म्हणजे त्यांच्या धंद्याला तुमचा नकार.

स्वतः बघा (5 मिनिटं)

तुमच्या स्वतःच्या आयुष्याचा 3-2-1 तपासा: तुमचे photos/documents किती जागी आहेत? Phone एकच जागा असेल, तर आज रात्री Google Photos/Drive चालू करा: 15 मिनिटांत तुमचं 3-2-1 चालू होईल. आणि एक पाऊल पुढे: Drive उघडून एक जुनी file **परत आणून** बघा; आता तुम्ही जगातल्या त्या मोजक्या लोकांत आहात ज्यांनी आपला restore **तपासला** आहे.

विचार करा

1. (derivation) तुमचा backup रोज रात्री 2 वाजता होतो. दुपारी 3 वाजता disk मेली. काय हरवलं? आणि यातून business चा एक निर्णय काढा: backup **किती वेळाने** घ्यावा हे कशावरून ठरतं?

उत्तर: रात्री 2 ते दुपारी 3 मधलं **सगळं:** 13 तासांच्या orders, नोंदी, chats. म्हणजे backup ची वारंवारता हा technology चा नाही, **business** चा निर्णय आहे: “आपल्याला किती तासांचं काम हरवणं परवडतं?” भाजी-दुकान: एक दिवस चालेल, रोज रात्री पुरे. UPI: एक second ही नाही; म्हणून तिथे प्रत्येक नोंद त्याच क्षणी अनेक जागी जाते (replication) आणि वर backups वेगळे. या “किती हरवणं चालेल” च्या मोजपट्टीवर तुमचा business कुठे आहे, हे एका ओळीत लिहा; तुमच्या engineer शी होणारं अर्ध बोलणं तिथून सुरू होईल. (आणि Part 4 इथे संपतो त्याच वाक्यावर ज्याने सुरू झाला: data हीच मालमत्ता; बाकी सगळं तिची राखण.)

PART 4 चा शेवट: नकाशा आणि पुढचं पाऊल

जे तुमच्याकडे आता आहे

DATABASE	data चा एकमेव दरबान: वेगळा, कायम चालणारा program; file नाही, server आहे
INDEX	लावलेली यादी + खुणा; 10 million मधून ~23 पावलांत; किंमत: लिहिणं हळू, जागा जास्त
QUERY / SQL आकार	database ला विचारलेला प्रश्न / tables ची भाषा खण (table), पिशवी (document), खुंटी (key-value); आकार कामावरून, fashion वरून नाही
TRANSACTION	"सगळं किंवा काहीच नाही" चा गठ्ठा; अर्धवट अवस्था अस्तित्वातच नाही (rollback)
LOCK / RACE	मोजकी वस्तू + अनेक हात = कुंडी; नाहीतर शर्यत आणि दोन तिकिटं एक खुर्ची
QUEUE / ASYNC	आत्ता नोंदवा, नंतर करा; waiter ची चिठ्ठी; पडलेली system = साठलेली रांग, हरवलेलं काही नाही
CACHE	महाग उत्तर जवळ ठेवणं; किंमत: शिळेपणा; दाखवणं शिळं चालतं, बदलणं ताजंच लागतं
BACKUP vs REPLICATION	काळ-प्रत (चुकीपासून वाचवते) vs जिवंत प्रत (machine मरण्यापासून; पण चूकही copy करते)
3-2-1	3 प्रती, 2 प्रकारच्या जागा, 1 दुसऱ्या गावात; आणि: न तपासलेला backup = backup नाहीच

एक परीक्षा, स्वतःसाठी

पुस्तक न उघडता, बोलून उत्तरं द्या:

1. Excel आणि database मध्ये खरा फरक काय? (नुसतं "मोठा" चालणार नाही; तीन तुटी सांगा.)
2. 10 million नोंदींमधून शोध 23 पावलांत कसा होतो?
3. पैसे पाठवताना "वजा झाले पण जमा नाही झाले" ही अवस्था का अस्तित्वात येऊ शकत नाही?
4. Backup असूनही company का बुडू शकते? (दोन कारणं.)

अडलात तर: 1 -> 6.1, 2 -> 6.2, 3 -> 6.4, 4 -> 6.7.

Part 5 मध्ये काय आहे

आता तुमच्याकडे पूर्ण machine आहे: recipe चालते (Part 1-2), जग जोडलेलं आहे (Part 3), आठवण पक्की आहे (Part 4). शेवटचा प्रश्न उरला, आणि तो थेट पैशाचा आहे: **हे सगळं लाखो लोकांसाठी, एकाच वेळेस, परवडणाऱ्या खर्चात कसं चालवायचं?** Cloud म्हणजे नक्की काय आणि त्याचं bill कसं वाचायचं, traffic अचानक दहापट झाला तर काय, जगभर वेगवान कसं राहायचं (CDN), आणि शेवटी: एक खरं product उघडून त्यात या पूर्ण पुस्तकाचा नकाशा बघणं. Part 5: जगाला SERVE करणं. शेवटचा आणि सगळ्यात मोठा part.

PART 4 चा MINI-GLOSSARY

3-2-1	प्रतींचा नियम: 3 प्रती, 2 जागा-प्रकार, 1 दूर (6.7)
async	आत्ता नोंदवा, नंतर करा (6.5)
backup	काळ-प्रत; मागे वळण्याची सोय (6.7)
cache	महाग उत्तर जवळ ठेवणं; शिळेपणाची किंमत (6.6)
database	data चा दरबान-server; file नाही (6.1)
document	पिशवीत-पिशवी आकार (Instagram post) (6.3)
index	लावलेली यादी + खुणा; शोधाचं इंजिन (6.2)
invalidation	cache कधी रिकामा करायचा हा अवघड प्रश्न (6.6)
key-value	खुंटी-tag आकार; OTP, sessions (6.3)
lock	कुंडी: एका वेळेस एक हात (6.4)
query	database ला विचारलेला प्रश्न (6.2)
race	दोन कामांची शर्यत; निकाल नशिबावर (6.4)
replication	जिवंत प्रत; चूकही copy करते (6.7)
restore	backup परत आणणं; न तपासलेला = नाहीच (6.7)
rollback	अर्धा गट्टा उलटा फिरवणं (6.4)
SQL	tables शी बोलायची भाषा (6.3)
staleness	शिळेपणा: cache मधलं जुनं उत्तर (6.6)
table	खण-आकार: ओळी x रकाने; हिशोबाचा राजा (6.3)
transaction	सगळं-किंवा-काहीच-नाही चा गट्टा (6.4)
queue / worker	कामांची रांग / ती उचलणारा program (6.5)